

MCP_OKF_GATEWAY_MEMO

MCP + OKF Gateway Groundwork — Go/No-Go Memo (Capabilities-Plan Task P4-T5'.1)

Doc type	Research + design memo (§11.4.150/ §11.4.99 deep multi-angle research). NOT an implementation — no gateway code lands from this document.
Track/branch	(T1/feature/helixllm-full- extension)
Author	Research subagent (T1/main), dispatched from conductor
Date / access date	2026-07-08
Reads first	docs/research/ 07.2026/00_master/ MASTER_IMPLEMENTATION_PLAN.md §6.4; docs/research/ 07.2026/02_vision_generative/ CAPABILITIES_MASTER_PLAN_v2.md Task P4-T5' (+ §1.1 “MCP — beta SDKs for the 2026-07-28 RC now exist”); docs/research/ 07.2026/00_master/ 03_open_clarifications.md C2; 05_mcp_acp_protocols.md (this directory, 2026-07-06 research report)
Does NOT duplicate	05_mcp_acp_protocols.md — that report already resolved the “which ACP” (Zed vs IBM vs A2A) and “which OKF” questions and is the canonical reference for ACP disambiguation, tool-calling normalization (§4), and capability advertisement (§5). This memo narrows to exactly two open

	items that report flagged UNCONFIRMED: (1) Go SDK Streamable-HTTP server transport production-readiness, (2) a fresh OKF confirmation, then renders the go/no-go this cycle needs.
Supersedes on the narrow point above	05_mcp_acp_protocols.md §1.3 (“Streamable-HTTP transport in the Go SDK... UNCONFIRMED — verify in go-sdk/mcp package docs before wiring”) and CAPABILITIES_MASTER_PLAN_v2.md §1.1’s framing that the answer depends on the 2026-07-28 RC. Both are now resolved — see §1 below.

0. TL;DR verdict

[GO] — the official Go MCP SDK (github.com/modelcontextprotocol/go-sdk) ships a production-grade Streamable-HTTP **server** transport (`mcp.StreamableHTTPHandler / mcp.NewStreamableHTTPHandler / mcp.StreamableServerTransport / mcp.StreamableHTTPOptions`) in its current **stable, non-prerelease** release **v1.6.1**, targeting the current stable MCP spec **2025-11-25**. This is *not* gated on the 2026-07-28 release candidate — the RC (`v1.7.0-pre.1`, still a GitHub prerelease as of this writing) only *extends* the same transport to additionally accept the new stateless wire protocol; the transport, its stateless option, its session handling, its DNS-rebinding/cross-origin protections, and its bearer-token auth integration already exist in the current stable release. §3 sketches a stateless-first gateway design built on this transport.

OKF conclusion: CONFIRMED, format-not-protocol. Google Cloud’s Open Knowledge Format (OKF, v0.1, 2026-06-12) is a vendor-neutral **content packaging format** (a directory of Markdown files with YAML frontmatter, linked into a knowledge graph via ordinary Markdown links) — it is not a wire protocol, not an LLM API, and shares no relationship with MCP. 03_open_clarifications.md C2 and 05_mcp_acp_protocols.md §3 already reached this conclusion on 2026-07-06; this memo re-verified it against the same primary source plus independent secondary coverage on 2026-07-08 and found nothing to correct. §2 below gives the concrete integration shape.

1. Deep research: Go MCP SDK current state (§11.4.150/§11.4.99)

1.1 Version state as of 2026-07-08

Queried directly against the GitHub API (gh api repos/modelcontextprotocol/go-sdk/releases, .../tags) — not a blog summary, the actual release/tag objects:

Tag	prerelease	Published	Notes
v1.7.0-pre.1	true	2026-06-24	RC track — “brings full support for protocol version 2026-07-28”; body text: “The streamable HTTP transport accepts requests at protocol version 2026-07-28 only when StreamableHTTPOptions.Stateless = true.”
v1.6.1	false (latest stable)	2026-05-22	Current production release
v1.6.0	false	2026-05-08	HTTP header standardization, cross-origin protection changes
v1.5.0	false	2026-04-07	OAuth + SSE transport enhancements
v1.4.1	false	2026-03-13	DNS-rebinding protection added; requires Go 1.25+

The README’s own version-compatibility table (fetched from the main branch, 2026-07-08) confirms:

SDK Version	Latest MCP Spec	All Supported MCP Specs
v1.7.0+	2026-07-28	2026-07-28, 2025-11-25*, 2025-06-18, 2025-03-26, 2024-11-05
v1.4.0 – v1.6.1	2025-11-25*	2025-11-25*, 2025-06-18, 2025-03-26, 2024-11-05

(* client-side OAuth experimental in this range.) **v1.6.1 already targets the current stable spec 2025-11-25** — it is not an RC-only artifact.

1.2 Streamable-HTTP server transport: confirmed present in v1.6.1, not RC-gated

Fetches the actual source file at the v1.6.1 git tag (not main, not a summary — the exact tagged commit): <https://raw.githubusercontent.com/modelcontextprotocol/go-sdk/v1.6.1/mcp/streamable.go>. It is a ~2,300-line file implementing, on the server side:

- type `StreamableHTTPHandler` struct — a full `http.Handler` (`ServeHTTP`), session-keyed (`sessions map[string]*sessionInfo`), with per-session idle-timeout logic (`startPOST/endPOST/stopTimer`).
- type `StreamableHTTPOptions` struct with fields: **Stateless bool** (“A stateless server does not validate the `Mcp-Session-Id` header, and uses a temporary session with default initialization parameters” — already present in v1.6.1, *not* new in the RC), `JSONResponse bool` (send `application/json` instead of `text/event-stream`), `Logger`, `EventStore` (stream-resumption persistence), `SessionTimeout`, `DisableLocalhostProtection`, `CrossOriginProtection`.
- func `NewStreamableHTTPHandler(getServer func(*http.Request) *Server, opts *StreamableHTTPOptions) *StreamableHTTPHandler`.
- type `StreamableServerTransport` struct with `Connect(ctx) (Connection, error)` and its own `ServeHTTP`, plus `streamableServerConn` implementing the transport’s `Connection` interface (`Read/Write/Close/SessionID`), per-stream SSE delivery (`stream type, formatEventID/parseEventID, acquireStream`), `servePOST/serveGET` request handling.
- DNS-rebinding protection (auto-enabled for localhost servers, per the code’s own comment “added in the 1.4.0 version of the SDK”) and an optional `http.CrossOriginProtection` wrapper.
- A separate auth package (github.com/modelcontextprotocol/go-sdk/auth) providing `TokenVerifier` (a caller-supplied `func(ctx, token, req) (*TokenInfo, error)`), `RequireBearerTokenOptions`, `func RequireBearerToken(verifier TokenVerifier, opts *RequireBearerTokenOptions) func(http.Handler) http.Handler` (standard Go middleware shape), and `ProtectedResourceMetadataHandler` for OAuth Protected Resource Metadata — i.e. bearer-token auth is a drop-in `http.Handler` wrapper, not something HelixLLM has to hand-roll.

This directly resolves 05_mcp_acp_protocols.md §1.3’s “UNCONFIRMED” flag: the SDK README excerpt that report fetched apparently did not enumerate the Streamable-HTTP type names (it only named Stdio/Command transports); the

actual tagged source shows the server transport is real, hardened, and has been production-shipping since at least v1.4.0 (DNS-rebinding protection dates the transport's existence to that release, 2026-02-27 per the earlier report's release table).

Client-side counterpart (`StreamableClientTransport`, `streamableClientConn`) is in the same file — the SDK ships both roles, matching HelixLLM's dual need (§1.4 of the 2026-07-06 report: gateway as MCP *server* exposing Helix tools, and as MCP *host/client* consuming CodeGraph/ OpenDesign).

1.3 Current MCP spec's server-transport story (stdio vs Streamable HTTP, stateless mode)

Fetches <https://modelcontextprotocol.io/specification/2025-11-25/basic/transports> directly (the current stable spec page, not a blog paraphrase). Confirmed:

- The spec defines exactly two standard transports: **stdio** and **Streamable HTTP** (custom transports are permitted but not standard).
- **Session management is already optional in the current stable spec**, not something the 2026-07-28 RC introduces: “A server using the *Streamable HTTP* transport **MAY** assign a session ID at initialization time... Servers that require a session ID **SHOULD** respond to requests without an MCP-Session-Id header... with HTTP 400.” A server that never assigns a session ID is spec-compliant today, in 2025-11-25. This means the stateless-first design recommendation from `05_mcp_acp_protocols.md` §1.2/§7 does not need to wait for the RC either — it is a legal posture under the *current* spec, and the RC formalizes/hardens it (removes the initialize handshake and the session header entirely, replaces server-initiated calls with multi-round-trip requests, adds server/discover).
- The spec **MUST**-level security requirements for Streamable HTTP servers: validate Origin (DNS-rebinding), bind to localhost when running locally, and “implement proper authentication for all connections” — exactly the three primitives the Go SDK ships out of the box (`DisableLocalhostProtection` defaulting to *enabled*, `CrossOriginProtection`, the auth package's bearer-token middleware).
- Confirmed the 2026-07-28 RC is still a release candidate, not finalized, as of 2026-07-08 (20 days out from its scheduled 2026-07-28 publish date; v1.7.0-pre.1 on the SDK side is tagged prerelease: `true`).

1.4 What this changes vs the capabilities plan's framing

`CAPABILITIES_MASTER_PLAN_v2.md` §1.1 and Task P4-T5' framed the go/no-go as depending on “whether the beta SDK for the 2026-07-28 RC” ships the transport. That framing is **narrower than the actual state**: the transport is not a beta-RC

feature at all — it is a **stable, already-shipping (since $\geq$ v1.4.0)**, **spec-2025-11-25-targeting** feature of the *current* production release, v1.6.1. The RC only adds an additional protocol-version acceptance path gated by the same `StreamableHTTPOptions.Stateless` flag that already exists. **This makes the GO verdict stronger, not conditional on the RC finalizing on schedule.**

2. OKF — re-verification and concrete integration shape

2.1 Re-verification (2026-07-08)

Refetched Google Cloud’s own announcement (cloud.google.com/blog/products/data-analytics/how-the-open-knowledge-format-can-improve-data-sharing/) directly and cross-checked against seven independent secondary sources surfaced by search (MarkTechPost, SearchEngineJournal, Flowtivity, Suganthan, StartupHub.ai, Medium $\times 2$). All converge on the same description, none mentions MCP, tool-calling, or any wire/transport protocol:

“OKF v0.1 represents knowledge as a directory of markdown files with YAML frontmatter, with a small set of agreed-upon conventions that let wikis written by different producers be consumed by different agents without translation.” — Google Cloud Blog

- **Requires exactly one thing per concept file:** a `type` field in YAML frontmatter. Everything else (what types exist, what other fields, section structure) is producer-defined.
- **Concepts link via ordinary Markdown links**, forming a knowledge graph richer than filesystem parent/child relationships.
- **No SDK, no runtime, no service required** — “just markdown,” “just files,” “just YAML frontmatter” (direct quotes from the announcement). Shippable in git, renderable on GitHub.
- Announced 2026-06-12 as v0.1.

Conclusion: the prior finding stands, uncorrected. OKF is a knowledge-content packaging format. It is category-mismatched against “protocols to integrate” (MCP/ACP/A2A) — there is nothing to “implement” as a wire protocol, and no UNCONFIRMED residue on what the operator’s “OKF” reference denotes: it demonstrably matches Google Cloud’s OKF (confirmed independently by `03_open_clarifications.md` C2’s operator-resolved clarification and by this fresh check). No honest-gap flag needed here — unlike the Go-SDK transport question, this one has no ambiguity left to resolve.

2.2 Concrete integration shape (as content, never as a protocol)

1. **On-disk store.** HelixLLM/HelixAgent's RAG/memory/Skills knowledge (runbooks, API references, metric definitions, capability manifests) is authored as an OKF-conformant directory: one Markdown file per concept, YAML frontmatter with (at minimum) type, linked via standard Markdown links. This is compatible with — and does not replace — the existing constitution-mandated Markdown-doc discipline (§11.4.44/§11.4.65 doc conventions); an OKF directory is simply a stricter, agent-consumable subset convention layered on top.
 2. **Served through MCP, not as a new protocol surface.** The gateway (§3) exposes the OKF directory via an MCP **resources** capability (`resources/list`, `resources/read`) — each OKF concept file becomes one MCP resource. This is the existing MCP resources primitive; no new transport or schema is invented for OKF.
 3. **Read AND write.** Per the Google Cloud framing (“agents read and update directly,” distinct from read-only RAG-chunk retrieval), the gateway MAY also expose an MCP **tool** (not a resource) for structured concept updates — e.g. `okf_upsert_concept(type, path, frontmatter, body)` — so an agent can maintain the knowledge graph in place rather than only retrieving from it. This is optional groundwork, not required for the P4-T5'.1 go/no-go; flag as a follow-up task if HelixMemory/RAG wiring (T7' in the capabilities plan) wants it.
 4. **Not a provider, not a model capability flag.** OKF never appears in the CONST-036/040 capability-flag surface (`/v1/models[].capabilities`, `MCP server/discover`, `ACP initialize`) — those advertise MCP/LSP/ACP/Embedding/RAG/Skills/Plugins, and OKF is *content consumed by* the RAG/Skills flags, not a flag itself.
-

3. GO — stateless-first MCP gateway design sketch

This is a design sketch to guide implementation, not implementation. Per the go/no-go in §1, the design commits to the official SDK's Streamable-HTTP server transport.

3.1 Two roles (unchanged from 05_mcp_acp_protocols.md §7, reconfirmed)

1. **Gateway-as-MCP-server:** HelixLLM/HelixAgent expose local capabilities — coder (repomap/tools), vision (VLM captioning/analysis), embeddings, translation (NLLB), STT/OCR, RAG/OKF resources — as MCP tools/resources over the SDK's `mcp.StreamableHTTPHandler`, so any MCP client

(Claude Code, Zed, etc.) can consume them remotely, plus `mcp.StdioTransport` for same-host subprocess clients.

2. **Gateway-as-MCP-host/client:** the gateway uses `mcp.StreamableClientTransport/mcp.CommandTransport` to reach downstream MCP servers (CodeGraph, OpenDesign, filesystem) and folds their tools/list into the canonical JSON-Schema tool layer (`05_mcp_acp_protocols.md` §4.3) so served local models can call them via constrained decoding.

3.2 Stateless-first HTTP surface

- Construct the handler as `mcp.NewStreamableHTTPHandler(getServer, &mcp.StreamableHTTPOptions{ Stateless: true, DisableLocalhostProtection: false, CrossOriginProtection: http.NewCrossOriginProtection() /* or equivalent explicit policy */, SessionTimeout: 0, Logger: <structured logger> })`.
- `Stateless: true` matches both (a) the 2025-11-25 spec’s legal “never assign a session ID” posture and (b) forward-compatibility with 2026-07-28’s removal of `initialize/session` headers entirely — the same flag gates both, per §1.1/§1.3.
- Any cross-tool-call state HelixLLM needs (e.g., a long-running vision/video job) is carried as an **explicit application-level handle** returned in a tool result and passed as an argument on the next call — never as protocol session state. This mirrors the RC’s own “explicit handles passed between tool calls” direction (`05_mcp_acp_protocols.md` §1.2) and is achievable today because it never depended on the RC — it is an application-layer choice available under the current stable spec’s optional session model.
- Keep `stdio` MCP servers (CodeGraph, OpenDesign) local-only, never bound to a network interface; only the Streamable-HTTP surface is a network listener.

3.3 Port + process placement

Following the established HelixLLM port allocation convention (`:18434` coder, `:18436` translation, `:18437` Whisper, `:18438` Tesseract, `:18439` vision-VLM, `:18440` RAG-TEI, `:18441` ACP → A2A, `:18442` image-gen, `:18443` video-gen — per `MASTER_IMPLEMENTATION_PLAN.md` §1.1): the MCP gateway HTTP listener is the next free port in that block (candidate `:18444`; confirm against the live port registry at implementation time — this memo does not reserve it). Runs as its own process/container (via the `containers` submodule, §11.4.76/§11.4.173), not inside the coder fleet’s container, so an MCP-gateway restart never disturbs the live coder.

3.4 Auth

- Bind the handler with the SDK's own `auth.RequireBearerToken(verifier, opts)` middleware wrapper — do **not** hand-roll bearer-token parsing. `verifier` is HelixLLM-supplied: validate against the existing JWT/OAuth stack config (§CONST-042 — secrets in `.env`, never in `.mcp.json` or hardcoded).
- `DisableLocalhostProtection` stays `false` (default-on DNS-rebinding protection) unless the gateway is deliberately bound to `0.0.0.0` for LAN-network-provider use (per `RESUME.md` Phase 3's "Network-provider (LAN/VPN)" landed capability) — in that case, add `CrossOriginProtection` and rely on the bearer-token layer as the real authorization boundary, never on network topology alone (§11.4.10).
- `stdio-transport` downstream servers (`CodeGraph`, `OpenDesign`) are never given network exposure and never carry the HTTP auth layer at all — they inherit host-process trust only.

3.5 §11.4.108 runtime signature (definition of done for this task, once implemented)

A single machine-checkable observable proving the gateway is active and working on a clean target: a real MCP client (not a stub) performs `tools/list` against the gateway's Streamable-HTTP endpoint and receives Helix's actual tool schemas (`repomap/vision/embeddings/translation` — not a hardcoded stub list, sourced live from the provider/capability registry per `CONST-036/040`), followed by one real `tools/call` round-trip that returns a genuine result (e.g. an embeddings vector for a fixed probe string) — captured as evidence exactly as `05_mcp_acp_protocols.md` §7's existing acceptance criterion and `CAPABILITIES_MASTER_PLAN_v2.md` Task P4-T5's acceptance line already specify. No new criterion is invented here; this memo only confirms the SDK choice the criterion depends on is sound.

4. Danger zones (§11.4.92 multi-pass — regression / cross-feature / security angles)

1. **Transport version drift.** The SDK's own version-compatibility table shows a hard cutover: `v1.6.1` → 2025-11-25 max; `v1.7.0+` → 2026-07-28 capable. Pinning to `v1.6.1` now is correct for GO-today, but the RC's wire-format changes (no `initialize`, `server/discover` instead, multi-round-trip requests replacing server-initiated calls, `subscriptions/listen` replacing notifications, `roots/sampling/logging` deprecated) are a **near-term breaking rewrite risk** if the gateway is built assuming 2025-11-25 semantics will

persist. Mitigation: build strictly to the stateless subset that is valid under *both* spec versions (§3.2) and re-run this memo's go/no-go check when v1.7.0 leaves prerelease, before bumping the pinned SDK version.

2. **Stateless vs stateful session handling.** Stateless: true simplifies the design but forfeits server-initiated requests/notifications entirely (per the SDK doc comment: "Any server->client request is rejected immediately as there's no way for the client to respond" in stateless mode). If any planned Helix MCP tool needs to push server-initiated progress notifications (e.g., a long video-gen job), stateless-only will not support it without the explicit-handle/ polling workaround in §3.2 — confirm no P4-T5' tool genuinely requires server-push before committing to Stateless: true universally; a mixed mode (stateful for specific long-running-job tools, stateless for the rest) may be needed and should be a follow-up design decision, not assumed here.
3. **Auth on an HTTP MCP server (§11.4.10).** Exposing Streamable HTTP over the network turns a previously host-local capability surface into a network-reachable one. The SDK's RequireBearerToken + ProtectedResourceMetadataHandler are available but not auto-configured — a misconfigured/omitted verifier, or reliance on DisableLocalhostProtection bypass for LAN convenience without a real token check, would silently turn the coder/vision/embeddings capabilities into an unauthenticated network service. This must be gated the same way CONST-042/§11.4.10 gate every other credential surface: verifier wired from .env-sourced secrets, never a stub/always-true verifier, and covered by a security test asserting an unauthenticated tools/call is rejected.
4. **Tool-schema exposure.** tools/list on an HTTP-reachable server broadcasts the *shape* of every internal capability (repomap paths, vision model identifiers, embeddings dimensions, translation language pairs) to any client that can reach the port and pass auth. This is a reconnaissance-surface risk distinct from the auth question in #3 — even an authenticated but over-broadly-scoped token could enumerate capabilities beyond what that caller should see. Mitigation direction (not designed here): scope tool visibility per verified token/role rather than exposing the full tool registry to every authenticated caller uniformly; track as a follow-up hardening task once the base gateway lands.
5. **Downstream MCP host-role trust boundary.** The gateway's *host/client* role (§3.1 item 2) launches/connects to CodeGraph and OpenDesign as subprocess stdio servers running with the gateway process's own privileges. A compromised or malicious downstream MCP server (third-party npm package, per §11.4.78's CodeGraph consumption path) has full access to whatever the gateway process can reach. This is not new to the MCP-gateway design specifically (CodeGraph is already consumed this way per §11.4.78), but consolidating *more* downstream servers behind one gateway process

concentrates the blast radius — worth a dedicated review when the host role is actually implemented, not assumed safe by analogy.

6. **2026-07-28 RC's SDK-version compatibility cliff.** Per §11.4.6, do not guess forward: this memo pins v1.6.1 as GO-today evidence; it explicitly does NOT recommend pre-adopting v1.7.0-pre.1 (still prerelease: true on GitHub as of 2026-07-08) for production wiring. Re-verify at RC finalization (scheduled 2026-07-28) before any version bump.

Danger zone count: 6.

Sources verified 2026-07-08

- Go MCP SDK releases (GitHub API, primary) — <https://api.github.com/repos/modelcontextprotocol/go-sdk/releases>
- Go MCP SDK tags (GitHub API, primary) — <https://api.github.com/repos/modelcontextprotocol/go-sdk/tags>
- Go MCP SDK v1.6.1 mcp/streamable.go source (raw, primary) — <https://raw.githubusercontent.com/modelcontextprotocol/go-sdk/v1.6.1/mcp/streamable.go>
- Go MCP SDK v1.6.1 auth/auth.go source (raw, primary) — <https://raw.githubusercontent.com/modelcontextprotocol/go-sdk/v1.6.1/auth/auth.go>
- Go MCP SDK README / version-compatibility table (raw, main) — <https://raw.githubusercontent.com/modelcontextprotocol/go-sdk/main/README.md>
- Go MCP SDK repo (overview) — <https://github.com/modelcontextprotocol/go-sdk>
- Go MCP SDK package docs — <https://pkg.go.dev/github.com/modelcontextprotocol/go-sdk/mcp>
- MCP specification 2025-11-25 — Transports (primary, current stable spec) — <https://modelcontextprotocol.io/specification/2025-11-25/basic/transports>
- MCP 2026-07-28 release-candidate announcement — <https://blog.modelcontextprotocol.io/posts/2026-07-28-release-candidate/>
- Google Cloud — Open Knowledge Format announcement (primary) — <https://cloud.google.com/blog/products/data-analytics/how-the-open-knowledge-format-can-improve-data-sharing/>
- MarkTechPost — OKF introduction — <https://www.marktechpost.com/2026/06/16/google-cloud-introduces-open-knowledge-format-okf-a-vendor-neutral-markdown-spec-for-giving-ai-agents-curated-context/>
- SearchEngineJournal — Google Cloud announces OKF — <https://www.searchenginejournal.com/google-cloud-announces-the-open-knowledge-format/579253/>
- Flowtivity — Google's OKF explained — <https://flowtivity.ai/blog/google-open-knowledge-format/>

- Suganthan — OKF blog — <https://suganthan.com/blog/open-knowledge-format/>
- StartupHub.ai — OKF explained 2026 — <https://www.startuphub.ai/ai-news/insights/2026/google-open-knowledge-format-okf-explained-2026>
- Medium (Tahir) — What is OKF — <https://medium.com/@tahirbalarabe2/what-is-open-knowledge-format-okf-270b20791802>
- Medium (Marc Bara) — Google's new format for agent context — <https://medium.com/@marc.bara.iniesta/googles-new-format-for-agent-context-a-standard-or-just-a-folder-82fb21d92041>
- Prior HelixCode research (context, not re-verified line-by-line, cross-checked where cited above) — docs/research/07.2026/05_mcp_acp_protocols/05_mcp_acp_protocols.md (2026-07-06); docs/research/07.2026/00_master/03_open_clarifications.md; docs/research/07.2026/02_vision_generative/CAPABILITIES_MASTER_PLAN_v2.md; docs/research/07.2026/00_master/MASTER_IMPLEMENTATION_PLAN.md